

# Ship Balancing With Astar

December 8<sup>th</sup>, 2025

## Team NovaVision AI

Suryateja Duvvuri <[sduvv003@ucr.edu](mailto:sduvv003@ucr.edu)>

Rakshan Rajesh Jagadeesan <[rraje008@ucr.edu](mailto:rraje008@ucr.edu)>

Lyda Taing <[ltain005@ucr.edu](mailto:ltain005@ucr.edu)>

Andrew Maciborski <[amaci093@ucr.edu](mailto:amaci093@ucr.edu)>

## Abstract

Mr. Keogh presented us a problem involving a large cargo ship which has to be balanced based on the regulations from the Coast Guard. The containers in the ship are allocated in a 8x12 grid and for every time the container is relocated, it costs time and money. The crane is located above the top left corner of the grid. The goal is to find out the minimum number of moves for the ship's crane to balance the containers in the ship.

## Elicitation

On November 8<sup>th</sup>, our team had a meeting on Zoom with Mr. Keogh. We compiled a list of questions in a slide to present during the meeting. The list of the questions were sent to an appropriate email address on November 2<sup>nd</sup> at 7:59 pm which include:

Crane function:

- 1) If the crane operator stops in the middle of the work (shift is over), does the system save the progress automatically so that when the next crane operator arrives, they can resume from the last move?
- 2) Should our program show the total estimated time per move to the operator, or only the total time for the full plan?
- 3) The slides mentioned the parked position for the ship is always [01,08], but what if the ship orientation is different at the dock? Does that change the parked position?
- 4) The slides mentioned that the cost model uses Manhattan Distance to calculate the cost, which is 1 minute per grid move. In the real world duration are the horizontal and vertical moves equal in time?
- 5) If a mistake is made, should the program let the operator reverse the last confirmed move, or does the system have to recalculate the entire plan using the current layout?

- 6) The cost of If the container was lifted and returned to the same spot, does it count as a move in a log file?

#### Log Files:

- 7) Does the system ask the operator to check the log file before saving, or does it skip and save automatically?
- 8) In a specific scenario, if the system crashes while keeping logs, should the half-filled log file be saved?
- 9) The slides mentioned the log files need to be saved, so should they be saved just locally on the COSTCO computer, or also saved to a cloud, or archived to a network drive for long-term use?
- 10) If the program starts and then shuts down without loading any manifest, should the system still generate the file with only the session start and stop timestamp?
- 11) If the program fails to save the files due to some error such as full disk or permission issue, how should the system handle this problem? Should it alert the operator and flag the operation until the issue is resolved.

#### Balance computation:

- 12) Should our program prioritize fewer moves, a shorter total time, or more even distribution across rows when there are multiple minimal-balance solutions?
- 13) The slide mentioned for the ships in column 6 is the midpoint, but what if the ship has an asymmetrical design? Is column 6 always used as the dividing line when determining the midpoint?
- 14) Should we always achieve the perfect balance, i.e,  $Pr-Sr = 0$ , or can the  $Pr-Sr$  value be 10% to be considered optimal and finish earlier?
- 15) When stacking the container, can we use the UNUSED cells (empty ones) as temporary positions while achieving the optimal arrangement, or are some empty cells reserved for safety?
- 16) In the event that there is no set of moves that satisfy the balance or the 10% threshold, should the system execute the best possible plan or should the operator need approval every time?

#### Manifest File:

- 17) If the manifest file becomes corrupted, then does the program attempt file recovery or reject the file and alert the captain?
- 18) In the edited manifest file, can the original row ordering be modified based on our convenience (the coordinates are correct), or should the original ordering be present?

- 19) The slide mentioned that the manifest files are saved with the ship's name, but what if there is a scenario where 2 ships have the same name? Then how does the program identify the correct manifest file?
- 20) What if the initial manifest file already contains balanced container coordinates? Then, should the system still create a move plan and ask the user to press Enter once to terminate the software, or should it just show a notice saying nothing needs to be done
- 21) Should the system prepare the file and depend on the operator to submit it manually, or should it automatically email the captain the OUTBOUND manifest once it is generated?
- 22) The example from the slide showed that the weight is measured in kilograms, should the system handle unit conversion?
- 23) What happens when the container coordinates are out of bounds?

Legal, Safety and Maintenance:

- 24) Does the format of logs or manifests for Coast Guard inspections have any regulatory requirements (for example, particular words or column alignment)?
- 25) What if there are new operators who don't know how to use the system? Then, should the system contain a beginner mode that helps the operator learn how the system works?
- 26) Who else can access the log and manifest files other than the crane operator?
- 27) In order to analyze operator efficiency later, should the program record the time it took for each ship to process from start to OUTBOUND?
- 28) From the example on Problem 3 overview, page 32, it shows the timestamp like "10 18 2023: 01:14", is the timestamp recorded with a specific time zone or is it based on a local time of the port?
- 29) When an emergency occurs during the move operation such as the container starts to slip, should the system have an emergency stop? If so, should the system pause instantly and require manager approval to resume?
- 30) The maximum weight of the container stated to be 99,999 kg, however, the crane weight limit was not mentioned in the overview slide. Should the system need to verify that the container weights do not exceed the limit of the crane load limit before scheduling the moves?

In the slide presentation we only include 22 questions that are more likely reasonable to ask for clarification and due to the given time for a meeting. We only selected the questions that are not highlighted in red. After the meeting with Mr. Keogh, we received a clarification from his responses on the scope of the problem which helped us to have a better understanding.

# Requirement Specification

On October 13, 2025, we had a briefing about the problem for port-balancing. We were given the specifications such as:

1. Ship's dimensions
2. Balance formula given by the law
3. Rules to move the crane
4. Requirements for outputs like generating a log file of operations and format of the manifest file.

Our program reads a manifest file and if it's valid, it should be able to go through the crane operations, use a search algorithm like A\* search, and output the minimum number of moves that gives balance. If there are any illegal moves like going out of bounds or trying to pick from an empty spot, the program would avoid them.

Here is a table of all the common acronyms we will use.

Table 1: Acronym List

|        |                                                                    |
|--------|--------------------------------------------------------------------|
| A*     | A Star search                                                      |
| NAN    | Not a number                                                       |
| I/O    | Input and output                                                   |
| 2D     | 2 dimension                                                        |
| $f(n)$ | Total estimation cost from start node to goal node.                |
| $g(n)$ | Current state cumulative cost from the start node to current node. |
| $h(n)$ | Heuristic cost from current node to goal node                      |

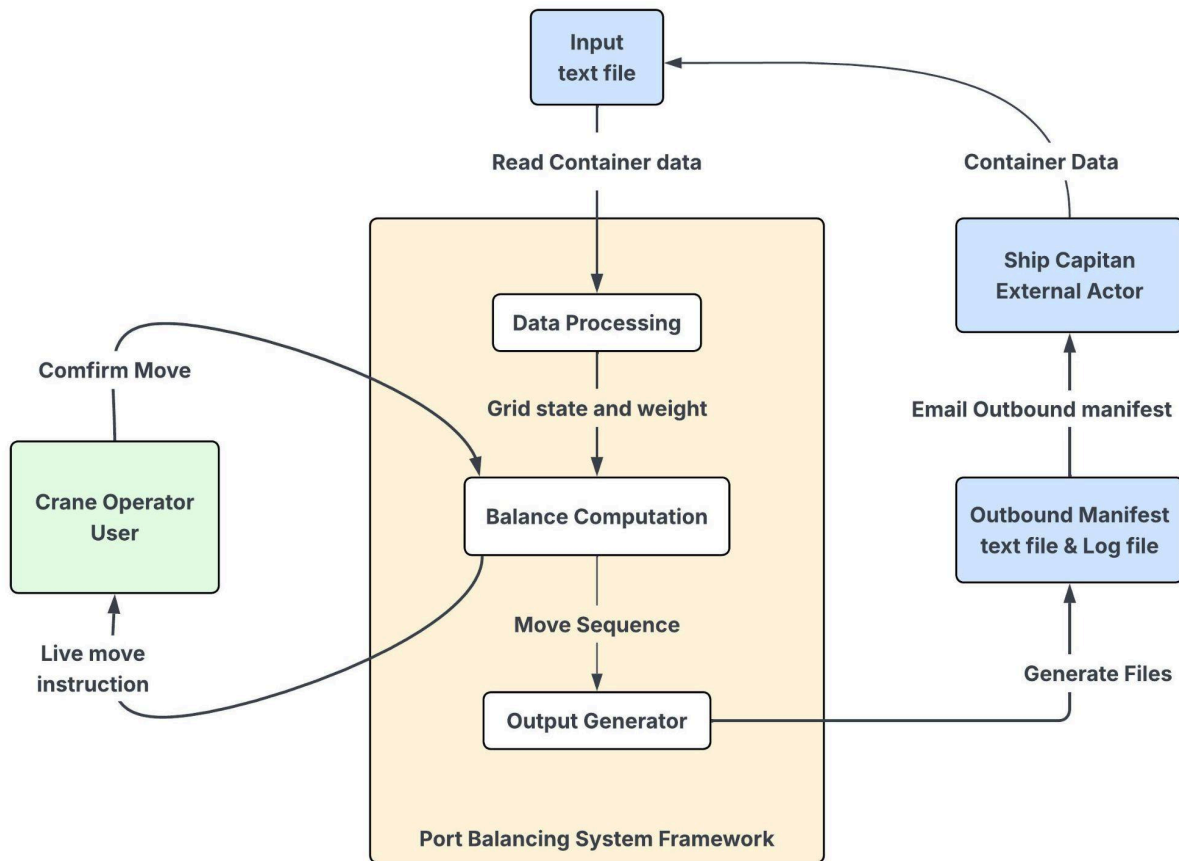


Figure 1: Port Balancing system context diagram.

A system context diagram shown in *Figure 1* illustrates a high level interaction between user and the software system. In this diagram, the user provides an input manifest file to the program in which the data parses the coordinate, weight and description. Then the system takes the preprocessing data and computes the balance. It validates the ship balance and generates the moves in order to achieve the balance. During the balance computation, it displays each move instruction to the crane operator and waits for them before moving to the next instruction. Once the ship achieves a balance state, an output generator compiles outbound files and log files which are later sent to the ship captain.

## System Functions

The system supports the following major functions:

- Verify the format of the input file
- Verifies the ship's balance with the given definition/formula
- Produces a log file consisting of operations

## Operating Environment

This program was developed using VScode with the official Python extension pack. It operates on both MacOS and windows. Standard libraries include os, re, sys, copy, heapq and datetime. The display is terminal-based, with instructions for a user to input a text file that contains the coordinates of points. The output display shows

The source code can be found at this [Repo](#). To run this program, follow the instructions below in your local terminal:

- `git clone https://github.com/dpxa/NovaVision-AI-proj3.git`
- `python main.py`
- Enter the name of the file: `test_cases/<filename>.txt`
- Press Enter each move executed.
- After successfully finishing the moves, the log file will be saved and sent back to the user.

## Design and Implementation Constraints

The application has the following constraints:

- The grid is created with the dimensions of 8 rows by 12 columns.
- The coordinates of the container are 1 by 1 base size with the origin [01, 01] at the bottom left.
- The coordinates are all non-negative values.
- Invalid slots that are not physically assigned to place the container are labeled as NAN in the user file.
- Empty slots are labeled as UNUSED which means the crane should not pick anything up from that coordinates.
- The maximum number of containers each ship can carry are 16 containers.
- Input and output must be in a text file.
- The system only handles one ship at a time.
- Crane always parks at [01,09] at the beginning and the end of each session.
- The metric of cost to move from one column/row to another is one minute. It is calculated by using Manhattan distance.

## Assumptions and Dependencies

The application has the following assumptions and dependencies:

- The input file is always correct with its format and data.
- The container can not float without any support from the bottom.
- The operator on the physical crane presses Enter key after they finish each move.
- The operators always move correctly according to the instructions from the solution provided by the system.

- The ship is considered balanced when there is no container, only one container, or has one container on each side of the deck.

## Design

Our system design is shown in *Figure 2*. The diagram illustrates how the main components of an application interact with one another. It shows the flow of data and how user signals interact with the system. It begins when the user provides the input file to the main module. The main function executes a read and parse manifest operation to create the grid state which is then entered into a decision block to check the balance of the ship. If it is not balance, the system compute the moves with A\* search algorithm. The solution is passed to the move instruction module which creates an interaction with a user. It displays each move instruction to the user and waits for a confirmation signal to move to the next move instruction until it reaches the end. Once it finishes, it checks if the ship reaches a balance state. Once it is balanced the system generates the outbound manifest and log file which is later sent to a ship captain.

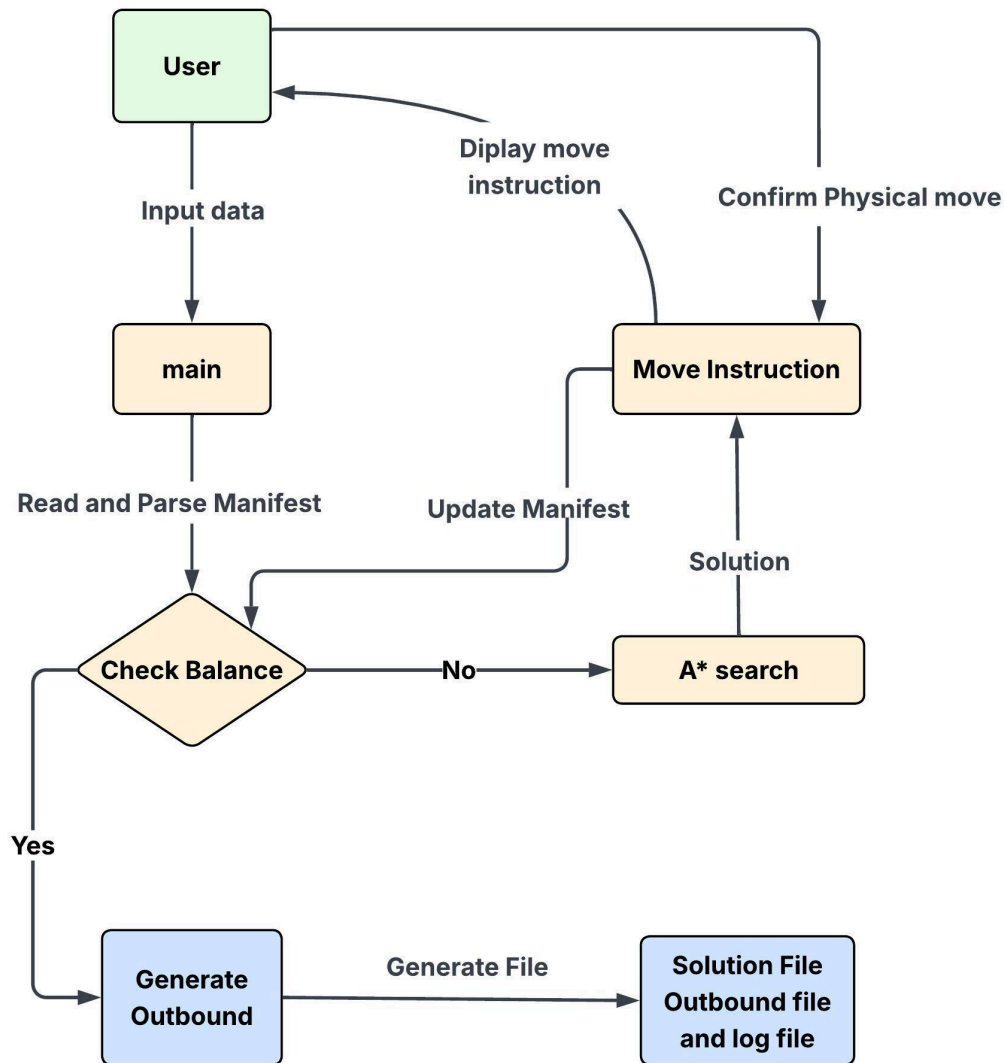


Figure 2: A top down description of the flow of application.

## Insight

### Algorithm Insights:

- The system has the ship with the grid of 8x12 where each cell either has an object of the Container class, none to indicate an empty cell or NAN if containers are not allowed to hold that cell. By establishing these operations, the container's structure, the crane position and what the crane holds are all considered as a state in which the search algorithm can be applied. The state changes if any of these parameters change. For instance, if two states have the same container structure but different crane positions, those states are different.



- The heuristic, which is  $h(n) = \frac{|Weight_{left} - Weight_{right}|}{2 \times MaxWeight}$ , gives a rough estimate on how many moves do we have based on the legal definition of balance which is  $|Ps-Pr| \leq 0.1(Ps+Pr)$ .
  - This heuristic is admissible since it does not overestimate the cost especially if moving the container changes the balance with its weight which should not make the container less balanced than its weight. Essentially, the denominator makes it so we scale the cost to be the minimum number of moves to balance the ship.
  - This is also helpful for going through states that are balanced because if the heuristic gives a huge number, this can indicate how much the ship's container is not balanced in which the heuristic will go through balanced states first.
  - This is good for the crane operation because the heuristic focuses on whether a certain operation would be efficient for getting the minimum amount of moves or making the container balanced.
- We implemented A\* algorithm to prioritize the states using the heuristic cost and actual cost from the crane operations. This wouldn't work if the container is either not balanced or if the crane is not used.

## Quantitative and Visual insights:

- We evaluated the test cases given from the lecture slides on the manifest files to understand the program's performance.
- A\* was good at solving small numbers of containers that were not balanced which was easy to make it balanced within minimum moves similar to the test cases.
- When the number of containers is big, the search algorithm explores fewer states because it avoids states that do not balance the ship.

## Performance and System Insights:

- One of the biggest factors of performance is performing deep copy of the Ship object for each state because this involves duplicating the state of the cells including the grid and the crane which means that the successor states branch out more.
- Some of the bottlenecks in the system include
  - calling the *copy.deepcopy()* in *operationPickup()* and *operationDrop()* for the crane.
  - *stateFetch()* is a hashing function for building tuples in rows which is needed to prevent repeated states from performing infinite loops.
  - For cells that contains NAN, illegal moves are eliminated which reduces the number of branches

- However, A\* handles these bottlenecks because the heuristic limits the depth for the search
  - Priority queue ordering is done efficiently by node comparisons.
  - Having an explored set as part of the search avoids revisiting states that have been seen before.

## Conceptual Reflection:

- While we are trying to solve the ship's balancing problem, it showed how A\* can solve problems that involve limits, rules and the cost of performing operations. We learned that reducing the number of branches is important for the search algorithm to efficiently search for moves and that labelling cells are important as well since that represents whether or not that cell can be touched or not which prevents illegal moves by the crane. Additionally, using the container's weights is helpful for the heuristic since that prioritizes which container to move for balancing the ship.

# Implementation

## System Overview

The ship balance system is designed to run on a standard low specification workstation such as low cost all-in-one PC. The software is implemented using Python for its development capabilities and supports a data structure for search algorithms. The main structure of the system is focused on the coordination data from the ship, search engine and the I/O handler.

## File structure and modules

The structure of the system is followed by 5 modules with a specific responsibility:

- **main.py** is a primary file that manages the user interface and handles input and output files. It manages the flow of data by calling each function from another file and passing data to check the balance and search for moves.
- **Ship.py** represents the physical attribute of the ship which includes the grid and crane position. It takes the input file and parses the data with the `read_manifest` function. It checks the state of the ship by compute the balance and generate move to achieve the balance with A\* search.
- **Container.py** is a data class that stores the attributes of a physical container based on the data in the input file. It consists of coordinates, weight, and description.
- **Node.py** is a class that represents the state of the ship. It specifically use in A\* search that computes the minimum cost of time to reach the state of balance ship. It allows the system to reconstruct the path of moves when the solution is found.

- **LogFile.py** is dedicated to writing a timestamp and detail of each event during a performing session. It then generates an output file upon the end of the session.

## Algorithm Implementation

The core algorithm of the system is relied on A\* search algorithm which is implemented in Ship.py. Dr. Keogh briefly mentioned A\* as a suggested algorithm during the briefing session. Additionally, A\* can guarantee the minimum time cost which provides the heuristic is admissible. The algorithm for A\* is implemented based on the following feature:

- **State Representation:** The grid of the ship is represented as a list of lists (2D mutable array) to track the state that has already been explored. During the moves calculation it stores in a list that is mutable. Then convert the grid into 2D tuples which are usable and stored in an explored set.
- **Priority Queue:** It organizes the nodes with the lowest total estimation cost  $f(n)$  at the front of the list, so that finding the node with the lowest cost in the frontier list will take a faster scan. The nodes are prioritized based on their total estimation cost  $f(n)$  where  $f(n) = g(n) + h(n)$ .
  - $g(n)$  is an actual cost from the start node to current node which calculated by using Manhattan Distance formula:  $g(n) = |x_1 - x_2| + |y_1 - y_2|$  and check if the path is free to move or else it will add a detour number of costs.
  - $h(n)$  is a heuristic cost from the current node to the final node which is calculated by using a custom formula: 
$$h(n) = \frac{|Weight_{left} - Weight_{right}|}{2 \times MaxWeight}$$
- **Transition:** The system generates the child node by simulating two crane operations that move a container from the grid to crane and move a container from crane to the grind. To make sure that there will be no corruption between one state to another, we use *copy.deepcopy*.

## Output Generation

At the end of the session, the system creates two output files:

- **Outbound Manifest:** is a file that derives from the input file with the modification of row index that represents the container that moves to achieve the balance of the ship. This file is generated by the *print\_outbound\_manifest* function in *main.py* that iterates through the final grid state. Then it saves in the test\_case folder with the <input file name>\_OUTBOUND.txt.

- **Audit Log:** is a file that records all the timestamps of the events during the operating session. The log file is generated by the *LogFile.py* module that runs in the background throughout the session and records all the significant events required for audit purposes.

## Testing

### Testing For Correct Program Behavior

To verify that the system behaves according to the specification requirement mentioned in briefing and description above, we conduct a test on error handling on input and output and the performance of A\* algorithm that corresponds to find the optimal cost in each session.

#### Error handling

Errors can happen due to the following reasons, but we handle the error with a proper message that is helpful for the user after ending the program:

- Incorrect file name or file format.
- Missing Python dependencies
- Invalid coordinate data
- No solution is found.

These handling techniques ensure the program runs smoothly and guide the user to fix the problem efficiently.

#### Quality of Solution

The quality of the solution is mainly focused on the minimum of operation time measured in minutes. Our system is guaranteed to find the path with the lowest cost if the solution exists. We use an admissible heuristic that estimates the cost based on weight difference which provides the efficient search and prevents unnecessary moves.

To verify our system, we demonstrate a test on ShipCase3.txt which is provided by Dr. Keogh with a known optimal route. The system checks all possible spots that satisfies the legal balance condition while simultaneously minimizing the total of operation cost.

Table 2 presents the process of A\* algorithm executes the trace for ShipCase3.txt. It shows the expansion order of the nodes based on the total estimate cost  $f(n)$ . In the table, the algorithm starts at the Home node and initially expand node [01,03] because it has the lowest  $f(n)$ . However, as it search the path to the goal node [01,07], node [01,06] became the lowest  $f(n)$  value, so it eventually expands Node [01,06] and continuously reaches the goal node.

Table 2: A\* search Trace for ShipCase3.txt.

| Node    | Cost from start g(n) | Cost to goal h(n) | Total f(n) = g(n) + (h) | Previous Node |
|---------|----------------------|-------------------|-------------------------|---------------|
| [09,01] | 0                    | 0.7               | 0.7                     | None          |
| [01,03] | 9                    | 0.3               | 9.3                     | [09,01]       |
| [01,02] | 10                   | 0.7               | 10.7                    | [01,03]       |
| [01,04] | 10                   | 0.7               | 10.7                    | [01,03]       |
| [01,05] | 11                   | 0.7               | 11.7                    | [01,03]       |
| [01,06] | 12                   | 0.3               | 12.3                    | [09,01]       |
| [01,07] | 13                   | 0.3               | 13.3                    | [01,06]       |

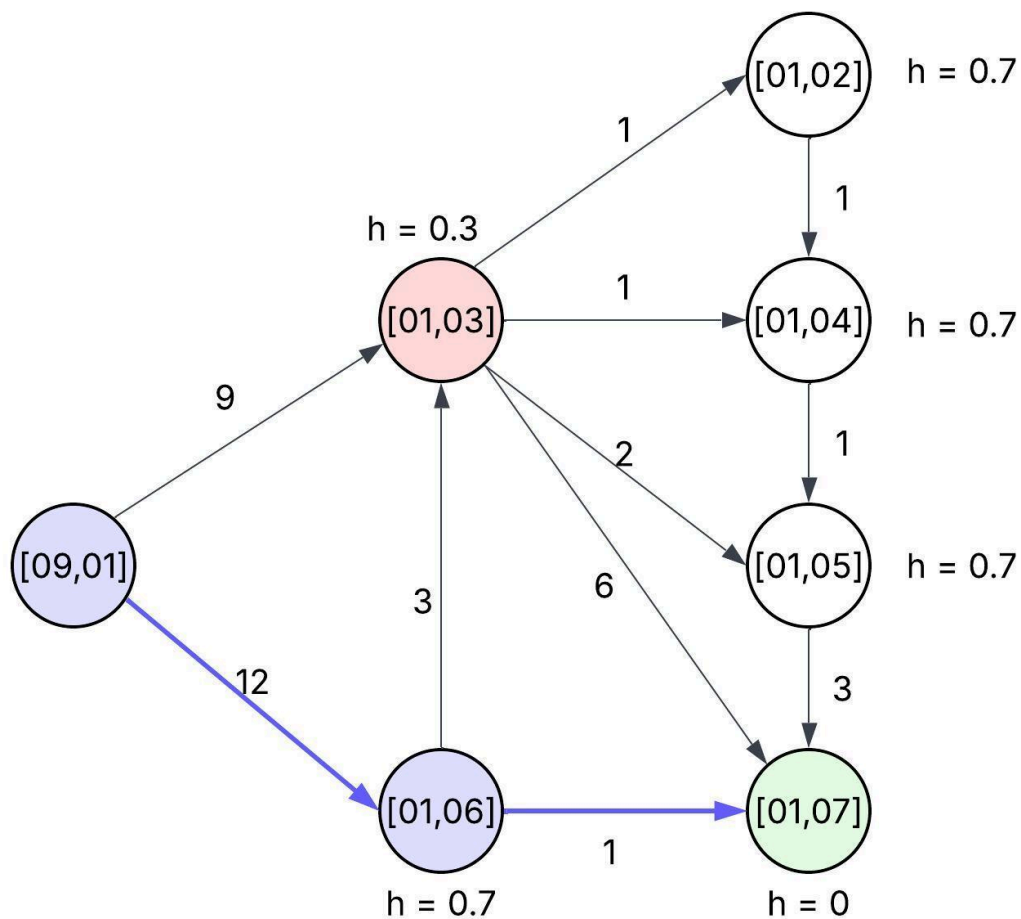


Figure 3: A\* Search Tree Graph for ShipCase3.txt

The graph in Figure 3 illustrates the branching behavior of A\* algorithm on ShipCase3.txt data. The search explored the node starting at [01,03] since it is the lowest cost. It follows this nature to

check every possible node until it breaks the lowest cost compared to its sibling node [01,06]. Eventually it moves to [01,06] and checks the path to the goal node [01,07].

| CRANE: HOME [9,1] |        |        |                 |        |        |             |        |        |        |        |        |        |
|-------------------|--------|--------|-----------------|--------|--------|-------------|--------|--------|--------|--------|--------|--------|
| EMPTY             |        |        |                 |        |        |             |        |        |        |        |        |        |
| 8                 | UNUSED | UNUSED | UNUSED          | UNUSED | UNUSED | UNUSED      | UNUSED | UNUSED | UNUSED | UNUSED | UNUSED | UNUSED |
| 7                 | UNUSED | UNUSED | UNUSED          | UNUSED | UNUSED | UNUSED      | UNUSED | UNUSED | UNUSED | UNUSED | UNUSED | UNUSED |
| 6                 | UNUSED | UNUSED | UNUSED          | UNUSED | UNUSED | UNUSED      | UNUSED | UNUSED | UNUSED | UNUSED | UNUSED | UNUSED |
| 5                 | UNUSED | UNUSED | UNUSED          | UNUSED | UNUSED | UNUSED      | UNUSED | UNUSED | UNUSED | UNUSED | UNUSED | UNUSED |
| 4                 | UNUSED | UNUSED | UNUSED          | UNUSED | UNUSED | UNUSED      | UNUSED | UNUSED | UNUSED | UNUSED | UNUSED | UNUSED |
| 3                 | UNUSED | UNUSED | UNUSED          | UNUSED | UNUSED | UNUSED      | UNUSED | UNUSED | UNUSED | UNUSED | UNUSED | UNUSED |
| 2                 | UNUSED | UNUSED | UNUSED          | UNUSED | UNUSED | UNUSED      | UNUSED | UNUSED | UNUSED | UNUSED | UNUSED | UNUSED |
| 1                 | NAN    | UNUSED | 101<br>Fish for | UNUSED | UNUSED | 40<br>Bikes | UNUSED | UNUSED | UNUSED | UNUSED | UNUSED | NAN    |
|                   | 1      | 2      | 3               | 4      | 5      | 6           | 7      | 8      | 9      | 10     | 11     | 12     |

Figure 4: Visual representation of the initial state of grid and container for ShipCase3.txt

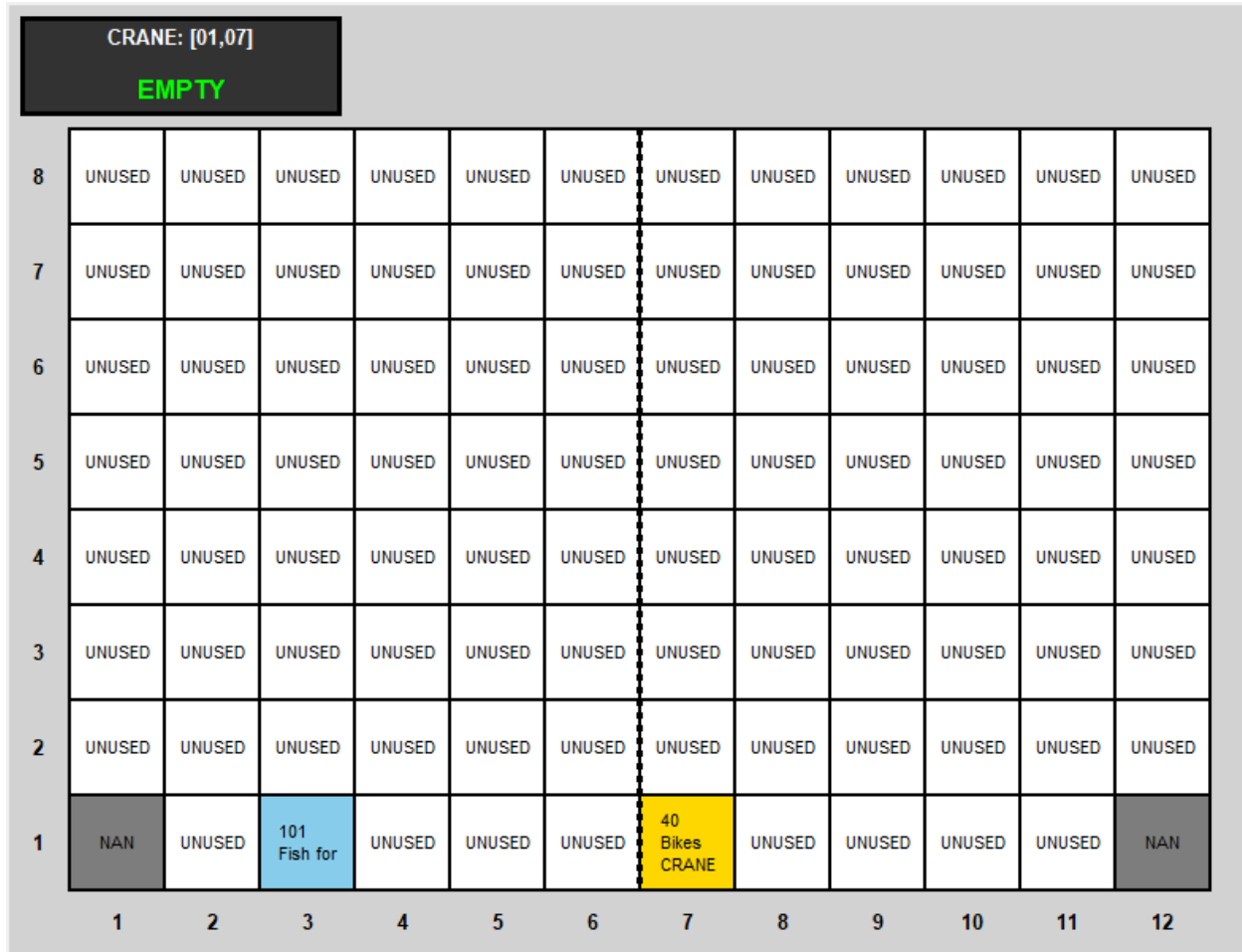


Figure 5: Visual Representation of the Balance state of grid and container for ShipCase3.txt

Intuitively, based on the visual representation from Figure 4, it is the lowest cost to move a container at [01,06] to [01,07] to satisfy the definition of legal balance that shows in Figure 5. Overall, the test results validate the system performance in three areas such as optimality, physical validation, and stability.

## Advanced Testing

In order to verify the reliability of the system, we tested a series of edge cases scenarios such as empty, one container, and legal balance threshold.

Table 3: A Cost Summary Table for Test cases.

| Test Cases    | Total Cost (minutes) | Number of step |
|---------------|----------------------|----------------|
| ShipCase1.txt | 0                    | 0              |
| ShipCase2.txt | 0                    | 0              |
| ShipCase3.txt | 26                   | 3              |

|               |    |   |
|---------------|----|---|
| ShipCase4.txt | 34 | 5 |
| ShipCase5.txt | 32 | 5 |
| ShipCase6.txt | 28 | 3 |

We test ShipCase1.txt data which contains no container. The image in Figure 6 below shows that there is no container in the ship which is considered as balance, therefore the cost for this case is 0 as shown on the summary Table 3. The system calculate\_balance() function returns True immediately and bypasses the search algorithm early, therefore it improves the run time of the system.

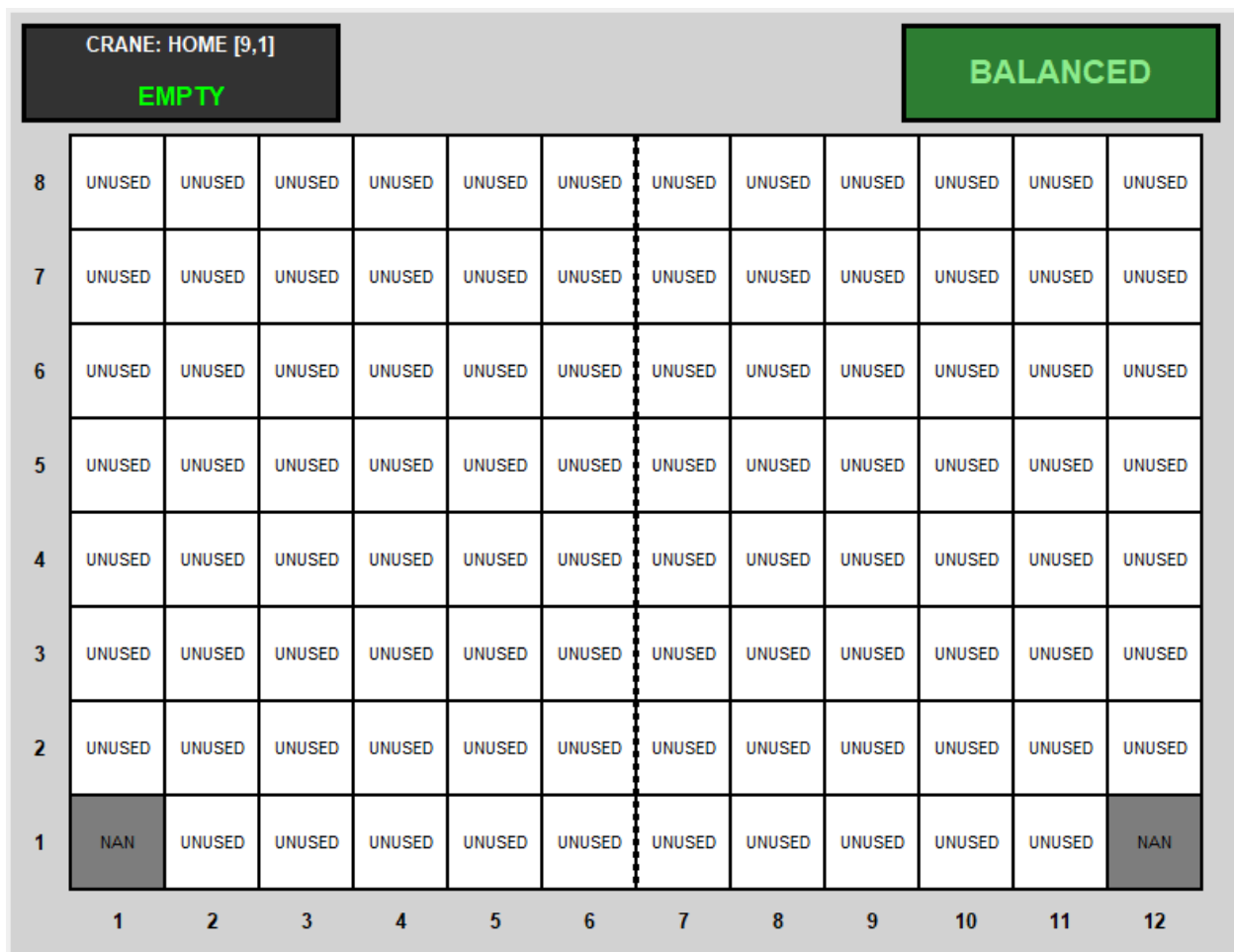


Figure 6: Visual representation of the grid and container for ShipCase1.txt

We also check the edge case condition where there is only one container on the ship that is provided by ShipCase2.txt file. The system generates the file and view in the GUI generator that shows in Figure 7. It shows that there is one container in the ship and by the condition of our constraint and assumption which set this case to be considered as balance, therefore the cost



for this case is 0 as shown on the summary Table 3. Similar to an empty ship case, the balance is flagged immediately to avoid search algorithm computation resources.

|                   |        |        |                 |        |        |          |        |        |        |        |        |        |
|-------------------|--------|--------|-----------------|--------|--------|----------|--------|--------|--------|--------|--------|--------|
| CRANE: HOME [9,1] |        |        |                 |        |        | BALANCED |        |        |        |        |        |        |
| EMPTY             |        |        |                 |        |        |          |        |        |        |        |        |        |
| 8                 | UNUSED | UNUSED | UNUSED          | UNUSED | UNUSED | UNUSED   | UNUSED | UNUSED | UNUSED | UNUSED | UNUSED | UNUSED |
| 7                 | UNUSED | UNUSED | UNUSED          | UNUSED | UNUSED | UNUSED   | UNUSED | UNUSED | UNUSED | UNUSED | UNUSED | UNUSED |
| 6                 | UNUSED | UNUSED | UNUSED          | UNUSED | UNUSED | UNUSED   | UNUSED | UNUSED | UNUSED | UNUSED | UNUSED | UNUSED |
| 5                 | UNUSED | UNUSED | UNUSED          | UNUSED | UNUSED | UNUSED   | UNUSED | UNUSED | UNUSED | UNUSED | UNUSED | UNUSED |
| 4                 | UNUSED | UNUSED | UNUSED          | UNUSED | UNUSED | UNUSED   | UNUSED | UNUSED | UNUSED | UNUSED | UNUSED | UNUSED |
| 3                 | UNUSED | UNUSED | UNUSED          | UNUSED | UNUSED | UNUSED   | UNUSED | UNUSED | UNUSED | UNUSED | UNUSED | UNUSED |
| 2                 | UNUSED | UNUSED | UNUSED          | UNUSED | UNUSED | UNUSED   | UNUSED | UNUSED | UNUSED | UNUSED | UNUSED | UNUSED |
| 1                 | NAN    | UNUSED | 232<br>Fish For | UNUSED | UNUSED | UNUSED   | UNUSED | UNUSED | UNUSED | UNUSED | UNUSED | NAN    |
|                   | 1      | 2      | 3               | 4      | 5      | 6        | 7      | 8      | 9      | 10     | 11     | 12     |

Figure 7: Visual representation of the grid and container for ShipCase2.txt

We check the legal balance threshold condition here. The ShipCase6.txt file has containers on the Port side and the Starboard side. The system obtains the file and views in the GUI generator, which is shown in Figure 8. A star search runs in our system, and we obtain a configuration that satisfies the 10% weight difference rule. The low final cost points to the fact that only small changes were required. This test shows us that the system also works on a case where there are multiple containers, and they must be arranged in a way to become balanced under the required constraint.

CRANE: HOME [9,1]

EMPTY

|   |        |        |        |         |         |         |        |        |        |        |         |        |
|---|--------|--------|--------|---------|---------|---------|--------|--------|--------|--------|---------|--------|
| 8 | UNUSED | UNUSED | UNUSED | UNUSED  | UNUSED  | UNUSED  | UNUSED | UNUSED | UNUSED | UNUSED | UNUSED  | UNUSED |
| 7 | UNUSED | UNUSED | UNUSED | UNUSED  | UNUSED  | UNUSED  | UNUSED | UNUSED | UNUSED | UNUSED | UNUSED  | UNUSED |
| 6 | UNUSED | UNUSED | UNUSED | UNUSED  | UNUSED  | UNUSED  | UNUSED | UNUSED | UNUSED | UNUSED | UNUSED  | UNUSED |
| 5 | UNUSED | UNUSED | UNUSED | UNUSED  | UNUSED  | UNUSED  | UNUSED | UNUSED | UNUSED | UNUSED | UNUSED  | UNUSED |
| 4 | UNUSED | UNUSED | UNUSED | UNUSED  | UNUSED  | UNUSED  | UNUSED | UNUSED | UNUSED | UNUSED | UNUSED  | UNUSED |
| 3 | UNUSED | UNUSED | UNUSED | UNUSED  | UNUSED  | UNUSED  | UNUSED | UNUSED | UNUSED | UNUSED | UNUSED  | UNUSED |
| 2 | UNUSED | UNUSED | UNUSED | UNUSED  | UNUSED  | UNUSED  | UNUSED | UNUSED | UNUSED | UNUSED | UNUSED  | UNUSED |
| 1 | NAN    | 61 Pig | 40 Owl | 334 Ewe | 333 Eel | 333 Cow | UNUSED | UNUSED | UNUSED | UNUSED | 900 Cat | NAN    |
|   | 1      | 2      | 3      | 4       | 5       | 6       | 7      | 8      | 9      | 10     | 11      | 12     |

Figure 8: Visual Representation of the Initial State of Grid and Container for ShipCase6.txt

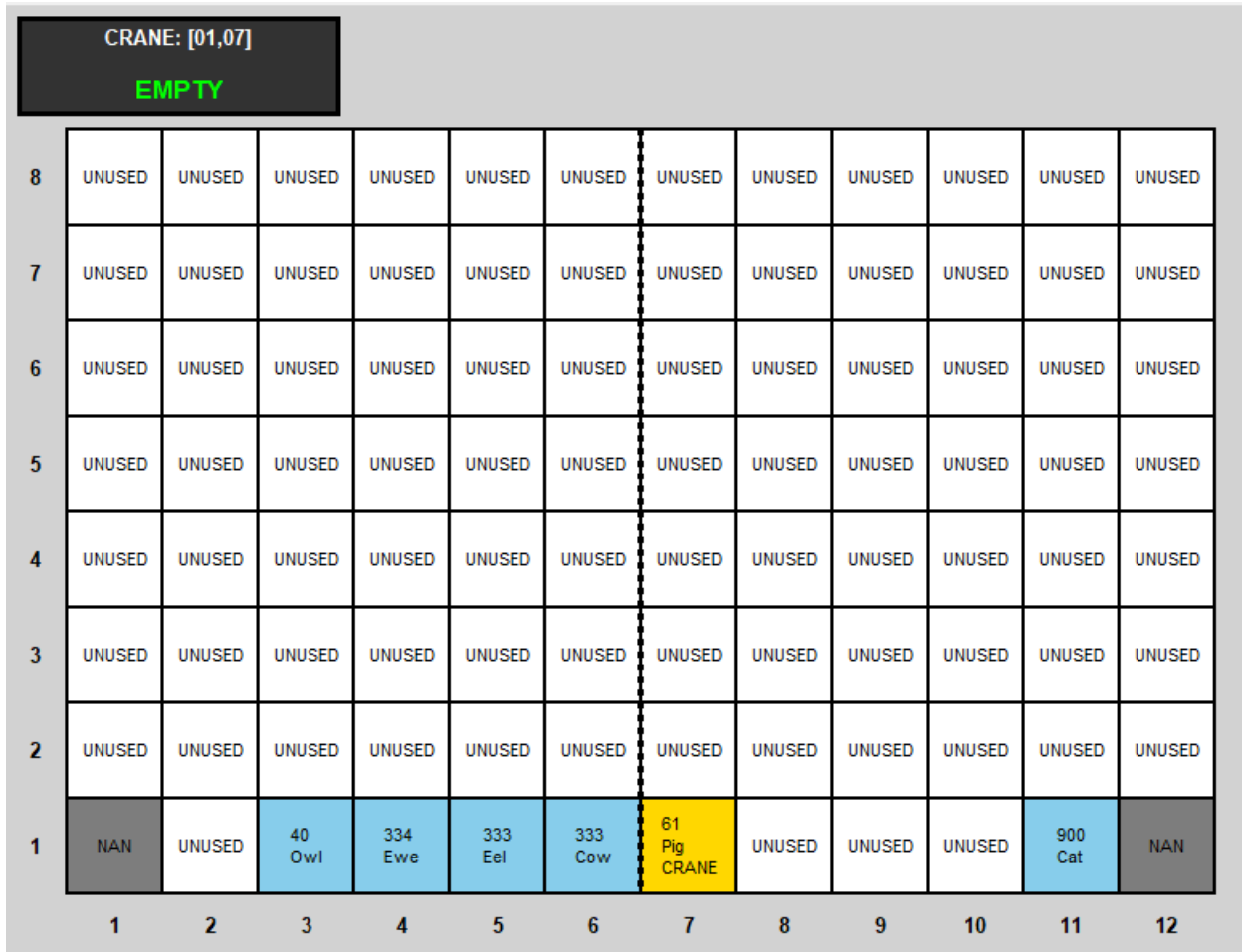


Figure 9: Visual Representation of the Balance State of Grid and Container for ShipCase6.txt

## Final Test

### Vertical Stack test:

In the final testing section, we are checking if our system can handle vertical stacking arrangements. For this testing, we use the *ShipCase4.txt*. There are 3 containers in this file, and they are placed on the port side (left). The starboard side is empty. Two containers are stacked on each other in column 2, and one container is in column 3, as shown in Figure 10. This tells us that the ship is unbalanced.

CRANE: HOME [9,1]

EMPTY

|   |        |                |              |        |        |        |        |        |        |        |        |        |
|---|--------|----------------|--------------|--------|--------|--------|--------|--------|--------|--------|--------|--------|
| 8 | UNUSED | UNUSED         | UNUSED       | UNUSED | UNUSED | UNUSED | UNUSED | UNUSED | UNUSED | UNUSED | UNUSED | UNUSED |
| 7 | UNUSED | UNUSED         | UNUSED       | UNUSED | UNUSED | UNUSED | UNUSED | UNUSED | UNUSED | UNUSED | UNUSED | UNUSED |
| 6 | UNUSED | UNUSED         | UNUSED       | UNUSED | UNUSED | UNUSED | UNUSED | UNUSED | UNUSED | UNUSED | UNUSED | UNUSED |
| 5 | UNUSED | UNUSED         | UNUSED       | UNUSED | UNUSED | UNUSED | UNUSED | UNUSED | UNUSED | UNUSED | UNUSED | UNUSED |
| 4 | UNUSED | UNUSED         | UNUSED       | UNUSED | UNUSED | UNUSED | UNUSED | UNUSED | UNUSED | UNUSED | UNUSED | UNUSED |
| 3 | UNUSED | UNUSED         | UNUSED       | UNUSED | UNUSED | UNUSED | UNUSED | UNUSED | UNUSED | UNUSED | UNUSED | UNUSED |
| 2 | UNUSED | 99<br>Tools fo | UNUSED       | UNUSED | UNUSED | UNUSED | UNUSED | UNUSED | UNUSED | UNUSED | UNUSED | UNUSED |
| 1 | NAN    | 200<br>Carpet  | 103<br>Tires | UNUSED | UNUSED | UNUSED | UNUSED | UNUSED | UNUSED | UNUSED | UNUSED | NAN    |
|   | 1      | 2              | 3            | 4      | 5      | 6      | 7      | 8      | 9      | 10     | 11     | 12     |

Figure 10: Visual representation of the initial grid and container for ShipCase4.txt

Our system runs the A\* search to make the ship balanced and correctly identifies that the 2 containers are stacked on each other, so its first move is to remove the top stacked container. The system performs a minimal set of moves in correct order (5 moves finishing in 34 minutes) to satisfy the 10% weight difference. The total cost (time) is 34 minutes. The system then moves the heaviest container (200 carpet) to the starboard side (left) and keeps the other 2 lighter containers stacked on each other on the port side (right). Figure 10 shows that the system makes a correct set of minimal moves (5 moves finishing in 34 minutes) to make the ship balanced even in conditions where containers are stacked on top of each other.

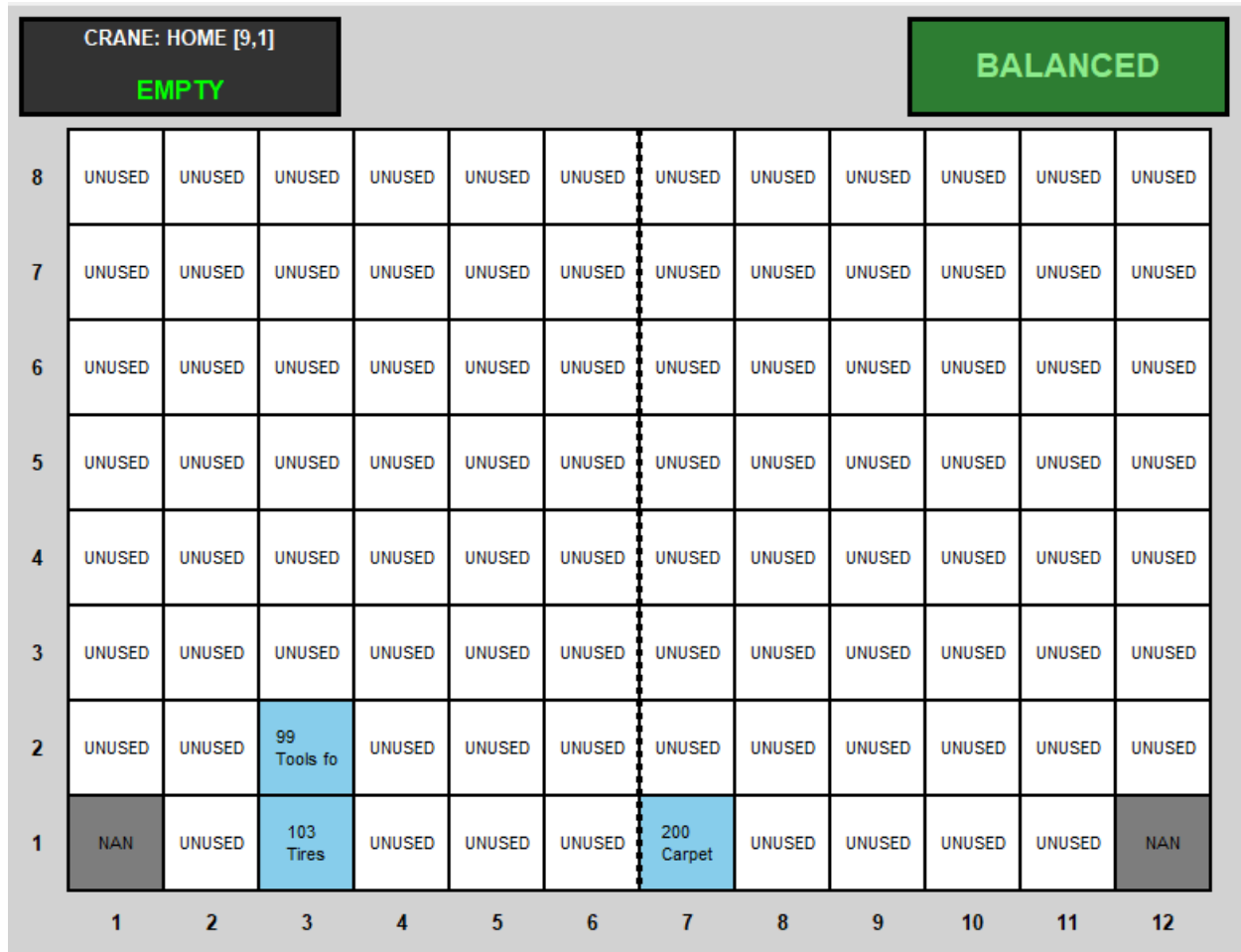


Figure 11: Visual representation of the final grid and container for ShipCase4.txt

## Stress test:

We are doing the stress test with ShipCase5.txt to check whether our system works under extreme conditions. We use ShipCase5.txt for our stress test because most of the containers are present on the port side (left). It contains one very heavy container (504 B). It has many empty columns. To get a balanced configuration, the system must do long-distance crane movement.

CRANE: HOME [9,1]

EMPTY

|   |        |          |          |         |          |          |        |          |        |        |        |        |
|---|--------|----------|----------|---------|----------|----------|--------|----------|--------|--------|--------|--------|
| 8 | UNUSED | UNUSED   | UNUSED   | UNUSED  | UNUSED   | UNUSED   | UNUSED | UNUSED   | UNUSED | UNUSED | UNUSED | UNUSED |
| 7 | UNUSED | UNUSED   | UNUSED   | UNUSED  | UNUSED   | UNUSED   | UNUSED | UNUSED   | UNUSED | UNUSED | UNUSED | UNUSED |
| 6 | UNUSED | UNUSED   | UNUSED   | UNUSED  | UNUSED   | UNUSED   | UNUSED | UNUSED   | UNUSED | UNUSED | UNUSED | UNUSED |
| 5 | UNUSED | UNUSED   | UNUSED   | UNUSED  | UNUSED   | UNUSED   | UNUSED | UNUSED   | UNUSED | UNUSED | UNUSED | UNUSED |
| 4 | UNUSED | UNUSED   | UNUSED   | UNUSED  | UNUSED   | UNUSED   | UNUSED | UNUSED   | UNUSED | UNUSED | UNUSED | UNUSED |
| 3 | UNUSED | UNUSED   | UNUSED   | UNUSED  | UNUSED   | UNUSED   | UNUSED | UNUSED   | UNUSED | UNUSED | UNUSED | UNUSED |
| 2 | UNUSED | UNUSED   | UNUSED   | UNUSED  | UNUSED   | UNUSED   | UNUSED | UNUSED   | UNUSED | UNUSED | UNUSED | UNUSED |
| 1 | NAN    | 101<br>Z | 100<br>X | 99<br>Y | 102<br>A | 504<br>B | UNUSED | 100<br>P | UNUSED | UNUSED | UNUSED | NAN    |
|   | 1      | 2        | 3        | 4       | 5        | 6        | 7      | 8        | 9      | 10     | 11     | 12     |

Figure 12: Visual representation of the initial grid and container for ShipCase5.txt

The system used A\* search to achieve balance. It performs the moves in correct order and performs a minimal set of moves (5 moves finishing in 34 minutes) required to satisfy the 10% weight difference. The total cost (time) is 34 minutes. The system moves the heaviest object to the starboard side (right). This test shows us that the system can handle conditions like significant weight disparities, a large number of unused columns, a bigger search space, and long crane movement. Even though the crane had to travel long distances to achieve balance, the system still gave an efficient solution.

CRANE: HOME [9,1]

EMPTY

BALANCED

|   |        |          |          |         |          |          |          |        |        |        |        |        |
|---|--------|----------|----------|---------|----------|----------|----------|--------|--------|--------|--------|--------|
| 8 | UNUSED | UNUSED   | UNUSED   | UNUSED  | UNUSED   | UNUSED   | UNUSED   | UNUSED | UNUSED | UNUSED | UNUSED | UNUSED |
| 7 | UNUSED | UNUSED   | UNUSED   | UNUSED  | UNUSED   | UNUSED   | UNUSED   | UNUSED | UNUSED | UNUSED | UNUSED | UNUSED |
| 6 | UNUSED | UNUSED   | UNUSED   | UNUSED  | UNUSED   | UNUSED   | UNUSED   | UNUSED | UNUSED | UNUSED | UNUSED | UNUSED |
| 5 | UNUSED | UNUSED   | UNUSED   | UNUSED  | UNUSED   | UNUSED   | UNUSED   | UNUSED | UNUSED | UNUSED | UNUSED | UNUSED |
| 4 | UNUSED | UNUSED   | UNUSED   | UNUSED  | UNUSED   | UNUSED   | UNUSED   | UNUSED | UNUSED | UNUSED | UNUSED | UNUSED |
| 3 | UNUSED | UNUSED   | UNUSED   | UNUSED  | UNUSED   | UNUSED   | UNUSED   | UNUSED | UNUSED | UNUSED | UNUSED | UNUSED |
| 2 | UNUSED | UNUSED   | UNUSED   | UNUSED  | UNUSED   | UNUSED   | UNUSED   | UNUSED | UNUSED | UNUSED | UNUSED | UNUSED |
| 1 | NAN    | 101<br>Z | 100<br>X | 99<br>Y | 102<br>A | 100<br>P | 504<br>B | UNUSED | UNUSED | UNUSED | UNUSED | NAN    |
|   | 1      | 2        | 3        | 4       | 5        | 6        | 7        | 8      | 9      | 10     | 11     | 12     |

Figure 13: Visual representation of the final grid and container for ShipCase5.txt

## Terminal Output:

```
Enter manifest file name: test_case/ShipCase5.txt
Log file was created: logs/ShipCase5_12_08_25_1702.txt

Ship is UNBALANCED.
Starting A* Search...

Solution was found, it will take 34 minutes and 5 moves.

1 of 5: Move from HOME [9,1] to [01,06], 13 minutes

2 of 5: Move container in [01,06] to [01,07], 1 minutes

3 of 5: Move from [00,06] to [01,08], 3 minutes

4 of 5: Move container in [01,08] to [01,06], 4 minutes

5 of 5: Move from [01,06] to HOME [9,1], 13 minutes

=====
Total Steps: 5
Total Time: 34 minutes
=====

Done! test_case/ShipCase5_OUTBOUND.txt was written to the desktop.

I have written an updated manifest to the desktop as test_case/ShipCase5_OUTBOUND.txt
Don't forget to email it to the captain.

Final manifest is here: test_case/ShipCase5_OUTBOUND.txt
Enter any comments to log file or press enter to exit:
```

*Figure 13: Terminal output for ShipCase5.txt*

## Conclusions

In this project, we used search algorithms to help balance a ship's container and learned how effective A\* is when it comes to making decisions with the given restrictions. When using the ship's structure and the crane's position and actions, we turn them into states and operators in which we were able to explore ways to fix the container's weights that are unbalanced. The heuristic influenced A\* to focus on moves that bring closer to balancing the ship instead of going through every combination with different moves. At the end, we learned the importance of designing heuristics and states along with the legal operations which helps the search algorithm produce better results.



## References

1. A star with heapq : <https://medium.com/@tahsinsoyakk/a-search-a-comprehensive-guide-8275ebdf8fae>
2. A star <https://www.redblobgames.com/pathfinding/a-star/implementation.html>
3. Timestamp : [https://www.w3schools.com/python/python\\_datetime.asp](https://www.w3schools.com/python/python_datetime.asp)
4. log file path: <https://docs.python.org/3/library/os.path.html> ,  
[https://www.w3schools.com/python/ref\\_func\\_open.asp](https://www.w3schools.com/python/ref_func_open.asp)
5. Copy module documentation: <https://docs.python.org/3/library/copy.html>
6. Tkinter for GUI: <https://docs.python.org/3/library/tkinter.html>
7. Test cases are provided by Dr. Keogh.